APE 5

Manejo de Archivos y Flujos de Datos (java.io)

Nombre: Anthony Daniel Armas Jaya
Curso: 1er Semestre
Carrera: Ingeniería en Software
Fecha: 06/11/2026
Repositorio de GitHub: https://github.com/Anthony-GTX/APE-5-INICIO

Se resolvieron 10 ejercicios en Java (6 de la primera parte y 4 de «Ejercicios APE 5»). Cada uno incluye enunciado, código, prueba de escritorio y captura de la ejecución.

N.º	Origen	Ejercicio	Clase Java
1	Parte 1 · Ejercicio 1	Crear un fichero en Java	CrearFichero.java
2	Parte 1 · Ejercicio 2	Escribir información en un fichero	EscribirFichero.java
3	Parte 1 · Ejercicio 3	Eliminar un fichero en Java	EliminarFichero.java
4	Parte 2 · Ejercicio 1	Registro de estudiantes y calificaciones	RegistroEstudiantes.java
5	Parte 2 · Ejercicio 2	Registro de productos con modo Append	RegistroProductos.java
6	Parte 2 · Ejercicio 3	Sistema de registro de ventas	RegistroVentas.java
7	Parte 3 · Ejercicio 1	Exploración y Gestión de Metadatos con File	ExplorarMetadatos.java
8	Parte 3 · Ejercicio 2	Escritura Estructurada por Líneas con BufferedWriter	EscribirMaterias.java
9	Parte 3 · Ejercicio 3	Control del Modo de Apertura (Append vs Sobrescribir)	AgregarMaterias.java
10	Parte 3 · Ejercicio 4	Ciclo de Vida Completo y Borrado Seguro	CicloVidaFichero.java

Ejercicio 1 · Crear un fichero en Java

Parte 1 · Ejercicio 1

Enunciado

Crea curso.txt con createNewFile(), informa si se creó o ya existía y muestra el nombre con getName().

Código (CrearFichero.java)

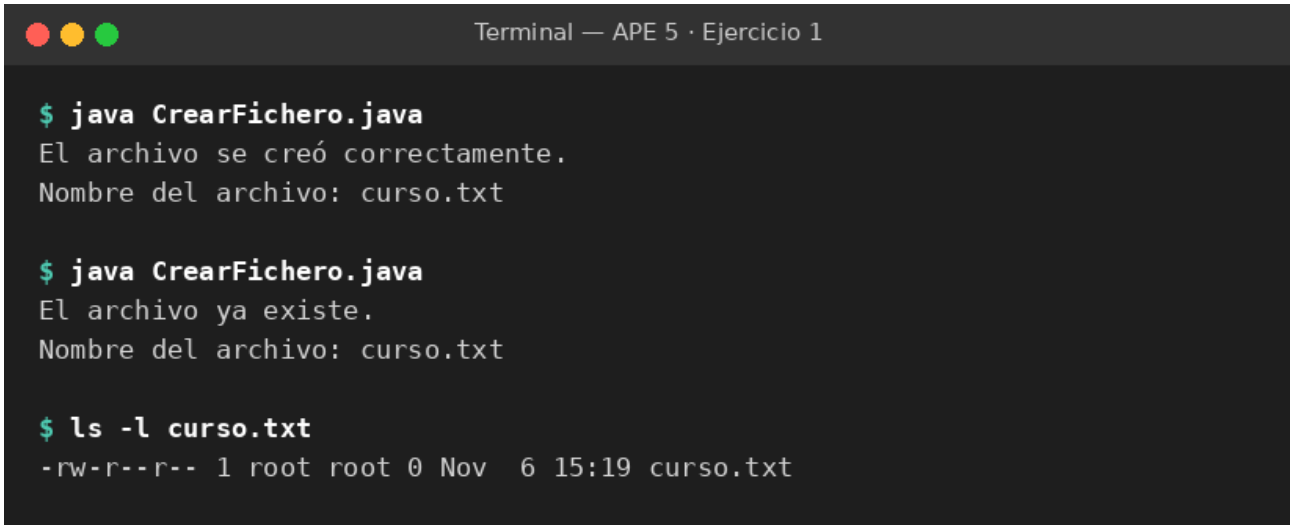
```
import java.io.File;
import java.io.IOException;

public class CrearFichero {
    public static void main(String[] args) {
        File archivo = new File("curso.txt");
        try {
            if (archivo.createNewFile()) {
                System.out.println("El archivo se creó correctamente.");
            } else {
                System.out.println("El archivo ya existe.");
            }
            System.out.println("Nombre del archivo: " + archivo.getName());
        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo: " + e.getMessage());
        }
    }
}
```

Prueba de escritorio

Ejecución	archivo	createNewFile()	Salida
1ª	curso.txt (no existe)	true	El archivo se creó correctamente.
2ª	curso.txt (ya existe)	false	El archivo ya existe.
Ambas	—	—	Nombre del archivo: curso.txt (getName())

Captura de la ejecución



Ejercicio 2 · Escribir información en un fichero

Parte 1 · Ejercicio 2

Enunciado

Lee un mensaje con Scanner y lo agrega al final de estudiantes.txt (existente) usando FileWriter con true.

Código (EscribirFichero.java)

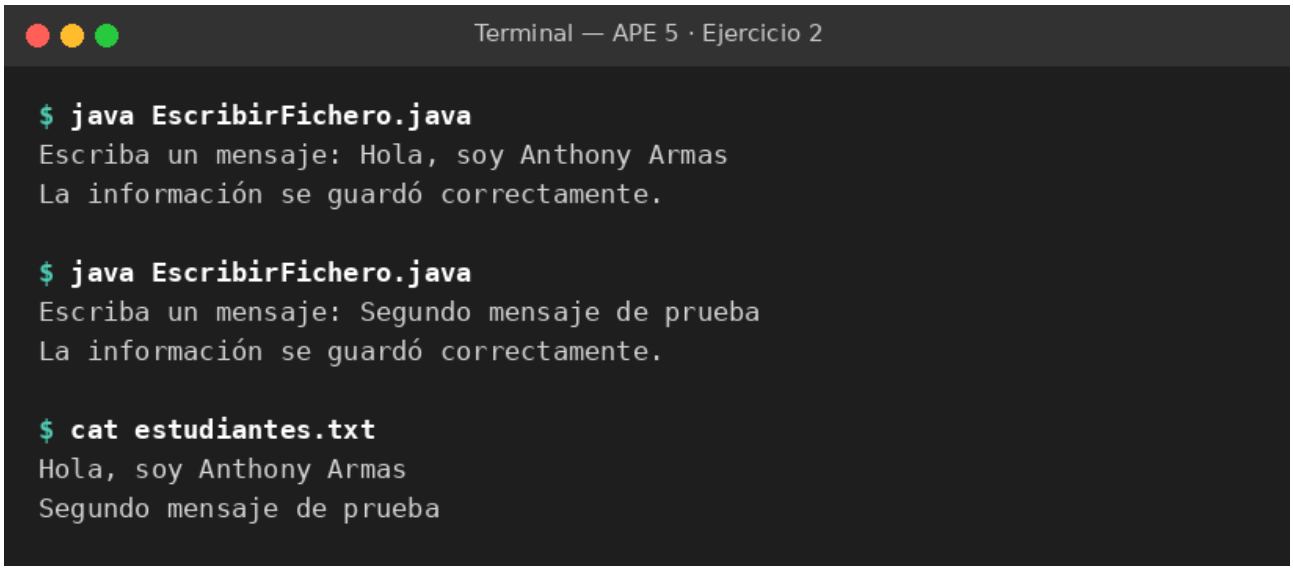
```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class EscribirFichero {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Escriba un mensaje: ");
        String mensaje = sc.nextLine();
        try {
            // true = modo append: conserva lo que ya tenía el archivo
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();
            System.out.println("La información se guardó correctamente.");
        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir: " + e.getMessage());
        }
        sc.close();
    }
}
```

Prueba de escritorio

Ejecución	mensaje (Scanner)	Modo FileWriter	Contenido de estudiantes.txt
1ª	Hola, soy Anthony Armas	append = true	Hola, soy Anthony Armas
2ª	Segundo mensaje de prueba	append = true	Hola, soy Anthony Armas / Segundo mensaje de prueba

Captura de la ejecución



Ejercicio 3 · Eliminar un fichero en Java

Parte 1 · Ejercicio 3

Enunciado

Elimina estudiantes.txt con delete() y muestra un mensaje según el resultado.

Código (EliminarFichero.java)

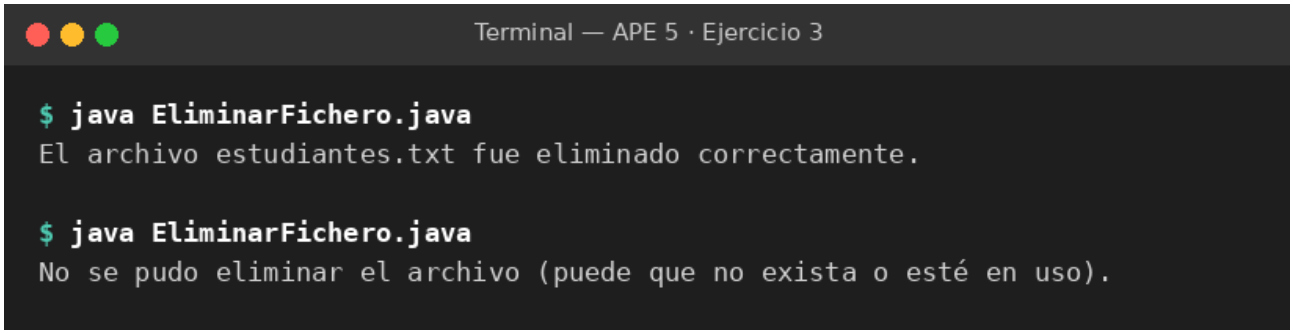
```
import java.io.File;

public class EliminarFichero {
    public static void main(String[] args) {
        File archivo = new File("estudiantes.txt");
        if (archivo.delete()) {
            System.out.println("El archivo " + archivo.getName() + " fue eliminado correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo (puede que no exista o esté en uso).");
        }
    }
}
```

Prueba de escritorio

Ejecución	¿Existe estudiantes.txt?	delete()	Mensaje
1ª	Sí	true	El archivo estudiantes.txt fue eliminado correctamente.
2ª	No	false	No se pudo eliminar el archivo (…)

Captura de la ejecución



Ejercicio 4 · Registro de estudiantes y calificaciones

Parte 2 · Ejercicio 1

Enunciado

Registra 3 estudiantes en estudiantes.txt (nombre y calificación, dos líneas por estudiante) con Scanner, BufferedWriter y FileWriter; el archivo se cierra en el bloque finally.

Código (RegistroEstudiantes.java)

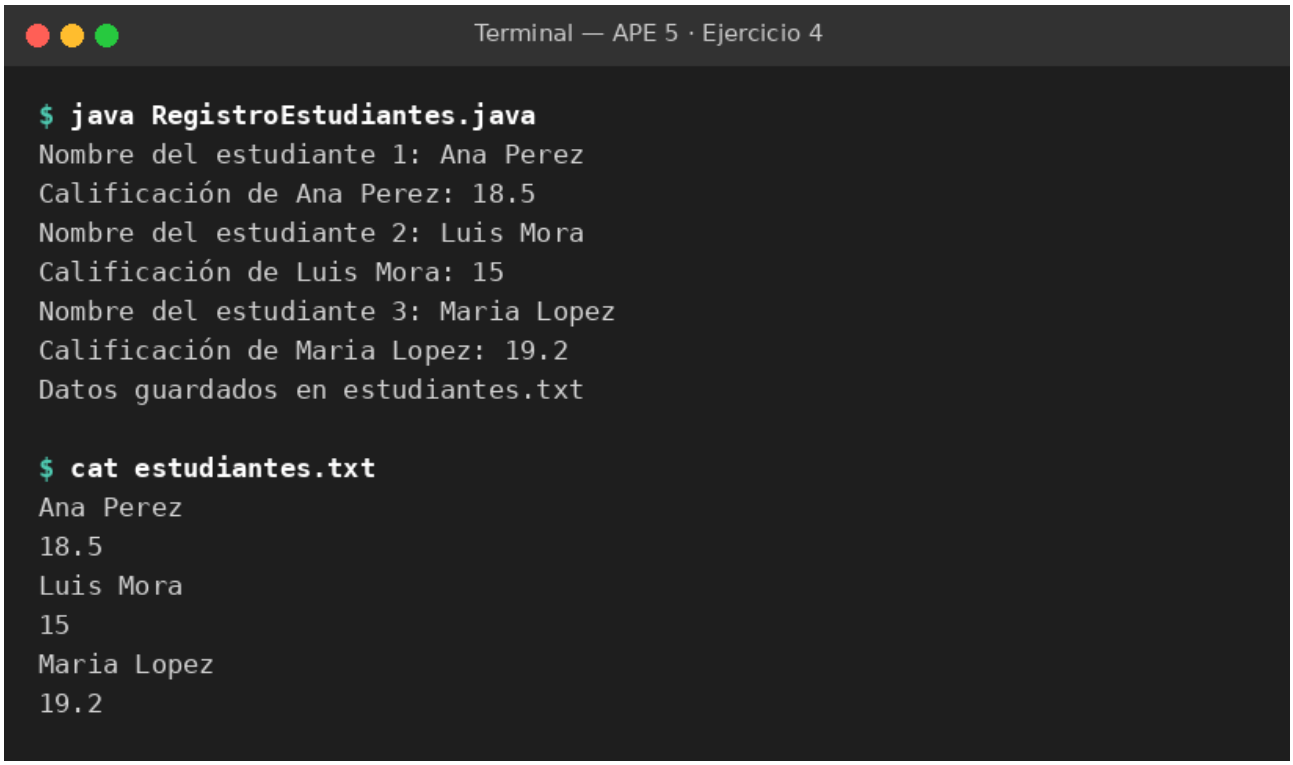
```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class RegistroEstudiantes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BufferedWriter bw = null;
        try {
            bw = new BufferedWriter(new FileWriter("estudiantes.txt"));
            for (int i = 1; i <= 3; i++) {
                System.out.print("Nombre del estudiante " + i + ": ");
                String nombre = sc.nextLine();
                System.out.print("Calificación de " + nombre + ": ");
                String nota = sc.nextLine();
                bw.write(nombre);   // línea 1
                bw.newLine();
                bw.write(nota);     // línea 2
                bw.newLine();
            }
            System.out.println("Datos guardados en estudiantes.txt");
        } catch (IOException e) {
            System.out.println("Error de archivo: " + e.getMessage());
        } finally {
            try {
                if (bw != null) bw.close();   // cierre correcto del archivo
            } catch (IOException e) {
                System.out.println("Error al cerrar: " + e.getMessage());
            }
            sc.close();
        }
    }
}
```

Prueba de escritorio

i	nombre	nota	Líneas escritas (write + newLine)
1	Ana Perez	18.5	Ana Perez ⏎ 18.5 ⏎
2	Luis Mora	15	Luis Mora ⏎ 15 ⏎
3	Maria Lopez	19.2	Maria Lopez ⏎ 19.2 ⏎
fin	—	—	finally → bw.close() (6 líneas en total)

Captura de la ejecución



Ejercicio 5 · Registro de productos con modo Append

Parte 2 · Ejercicio 2

Enunciado

Pide nombre, precio y cantidad, calcula precio × cantidad y lo guarda en productos.txt en modo append (no borra registros anteriores).

Código (RegistroProductos.java)

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

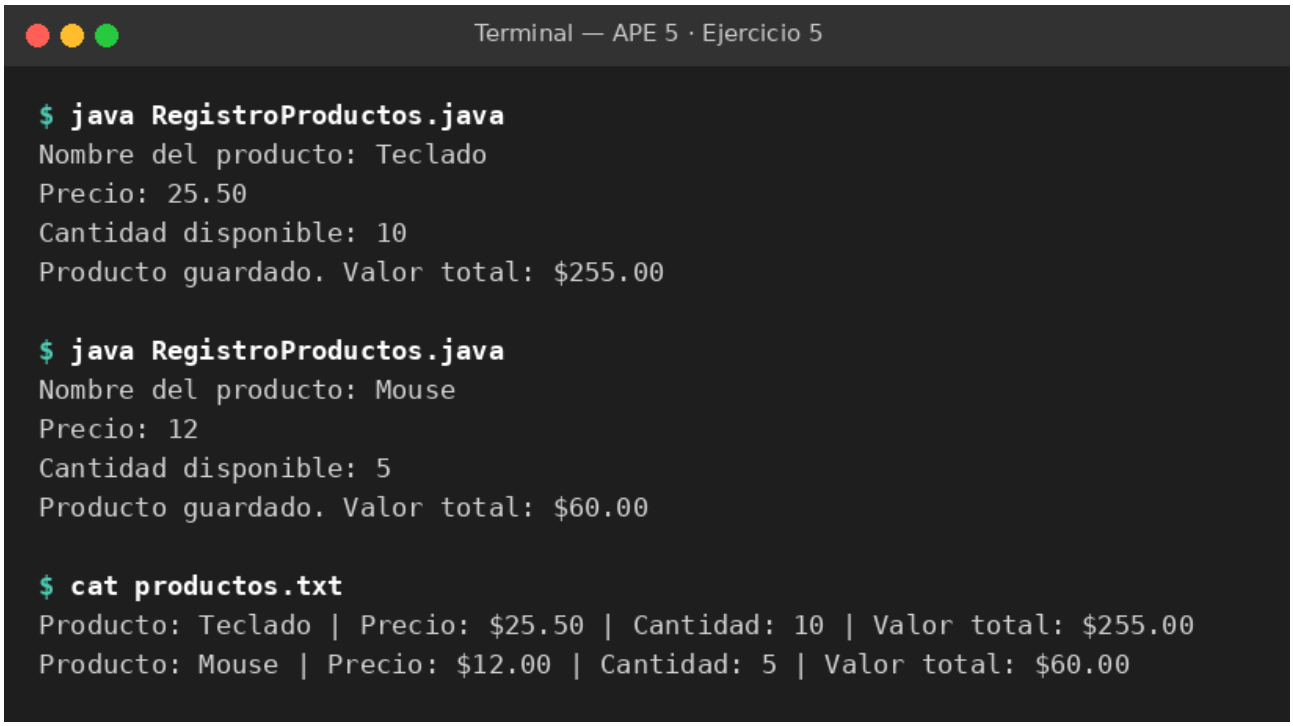
public class RegistroProductos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nombre del producto: ");
        String nombre = sc.nextLine();
        System.out.print("Precio: ");
        double precio = Double.parseDouble(sc.nextLine());
        System.out.print("Cantidad disponible: ");
        int cantidad = Integer.parseInt(sc.nextLine());
        double total = precio * cantidad;

        // try-with-resources cierra el archivo automáticamente; true = append
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("productos.txt", true))) {
            bw.write("Producto: " + nombre
                    + " | Precio: $" + String.format("%.2f", precio)
                    + " | Cantidad: " + cantidad
                    + " | Valor total: $" + String.format("%.2f", total));
            bw.newLine();
            System.out.println("Producto guardado. Valor total: $" + String.format("%.2f", total));
        } catch (IOException e) {
            System.out.println("Error de archivo: " + e.getMessage());
        }
        sc.close();
    }
}
```

Prueba de escritorio

Ejecución	nombre	precio	cantidad	total = precio × cantidad
1ª	Teclado	25.50	10	25.50 × 10 = 255.00
2ª	Mouse	12	5	12 × 5 = 60.00

Captura de la ejecución



Ejercicio 6 · Sistema de registro de ventas

Parte 2 · Ejercicio 3

Enunciado

Registra ventas hasta escribir «fin»; calcula el subtotal, aplica 10 % de descuento si es ≥ $100, muestra el total a pagar, guarda cada venta en ventas.txt (append) y al final guarda el total acumulado.

Código (RegistroVentas.java)

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class RegistroVentas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double acumulado = 0;

        try (BufferedWriter bw = new BufferedWriter(new FileWriter("ventas.txt", true))) {
            while (true) {
                System.out.print("Nombre del producto (o 'fin' para terminar): ");
                String nombre = sc.nextLine();
                if (nombre.equalsIgnoreCase("fin")) {
                    break;
                }
                System.out.print("Precio: ");
                double precio = Double.parseDouble(sc.nextLine());
                System.out.print("Cantidad vendida: ");
                int cantidad = Integer.parseInt(sc.nextLine());

                double subtotal = precio * cantidad;
                double descuento = 0;
                if (subtotal >= 100) {
                    descuento = subtotal * 0.10;
                }
                double totalPagar = subtotal - descuento;
                acumulado += totalPagar;

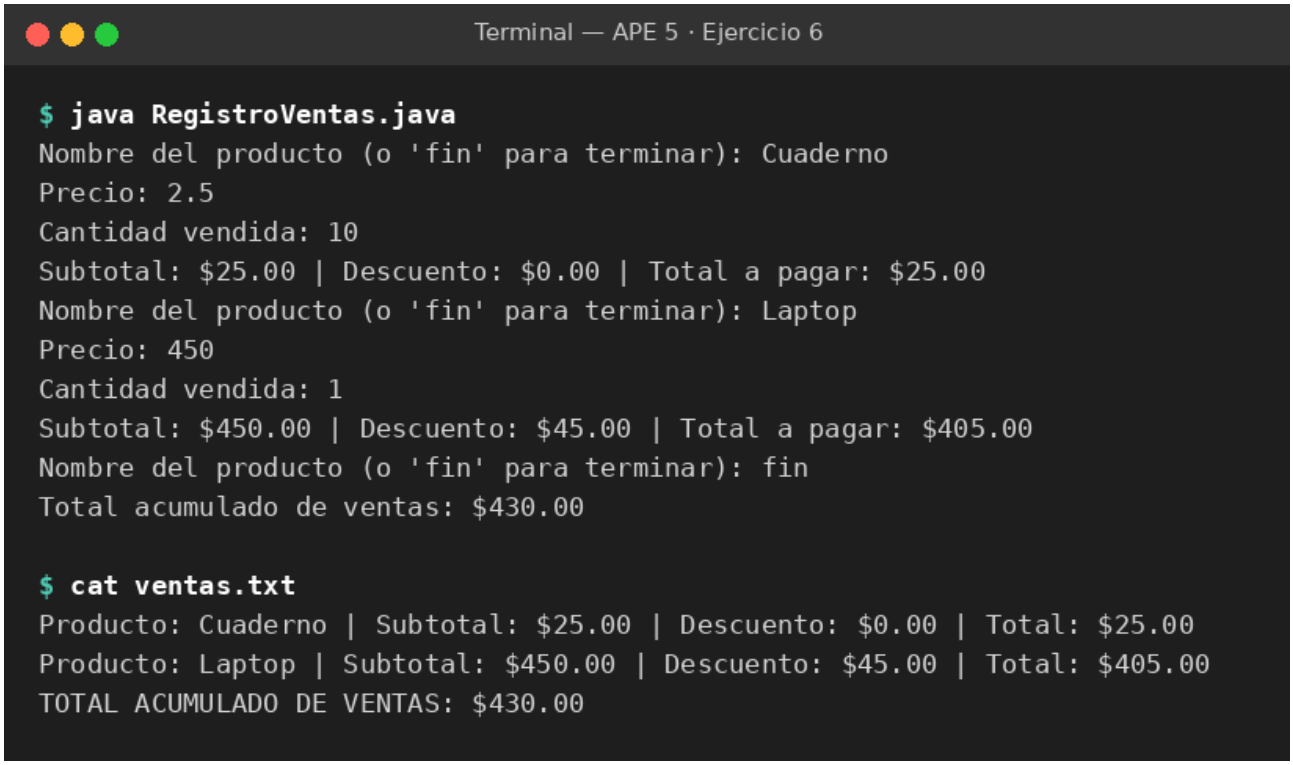
                System.out.println("Subtotal: $" + String.format("%.2f", subtotal)
                        + " | Descuento: $" + String.format("%.2f", descuento)
                        + " | Total a pagar: $" + String.format("%.2f", totalPagar));

                bw.write("Producto: " + nombre
                        + " | Subtotal: $" + String.format("%.2f", subtotal)
                        + " | Descuento: $" + String.format("%.2f", descuento)
                        + " | Total: $" + String.format("%.2f", totalPagar));
                bw.newLine();
            }
            bw.write("TOTAL ACUMULADO DE VENTAS: $" + String.format("%.2f", acumulado));
            bw.newLine();
            System.out.println("Total acumulado de ventas: $" + String.format("%.2f", acumulado));
        } catch (IOException e) {
            System.out.println("Error de archivo: " + e.getMessage());
        }
        sc.close();
    }
}
```

Prueba de escritorio

Iter.	nombre	precio	cant.	subtotal	descuento	total	acum.
0	—	—	—	—	—	—	0.00
1	Cuaderno	2.5	10	25.00	0.00 (<100)	25.00	25.00
2	Laptop	450	1	450.00	45.00 (10 %)	405.00	430.00
3	fin	—	—	—	—	—	430.00 → break

Captura de la ejecución



Ejercicio 7 · Exploración y Gestión de Metadatos con File

Parte 3 · Ejercicio 1

Enunciado

Basándote en los conceptos de rutas y abstracción de la sesión, escribe un programa en Java que utilice la clase File (perteneciente al paquete java.io) para interactuar con la ruta "datos/prueba.txt". Tu solución debe cumplir estrictamente con lo siguiente:

1. Comprobación previa: Utiliza el método adecuado (exists()) para verificar si la ruta apunta a un recurso disponible en el disco.

2. Creación controlada: Si el recurso no existe, intenta crearlo mediante el método correspondiente (createNewFile()). Como esta operación puede producir un error de E/S, debes gestionarlo obligatoriamente manejando la excepción IOException (mediante un bloque try-catch).

3. Extracción de información: Tanto si el archivo ya existía como si se acaba de crear, utiliza los métodos de consulta para mostrar por consola el último nombre de la ruta (getName()) y la ruta completa o relativa con la que se construyó el objeto (getPath()).

Código (ExplorarMetadatos.java)

```
import java.io.File;
import java.io.IOException;

public class ExplorarMetadatos {
    public static void main(String[] args) {
        File archivo = new File("datos/prueba.txt");

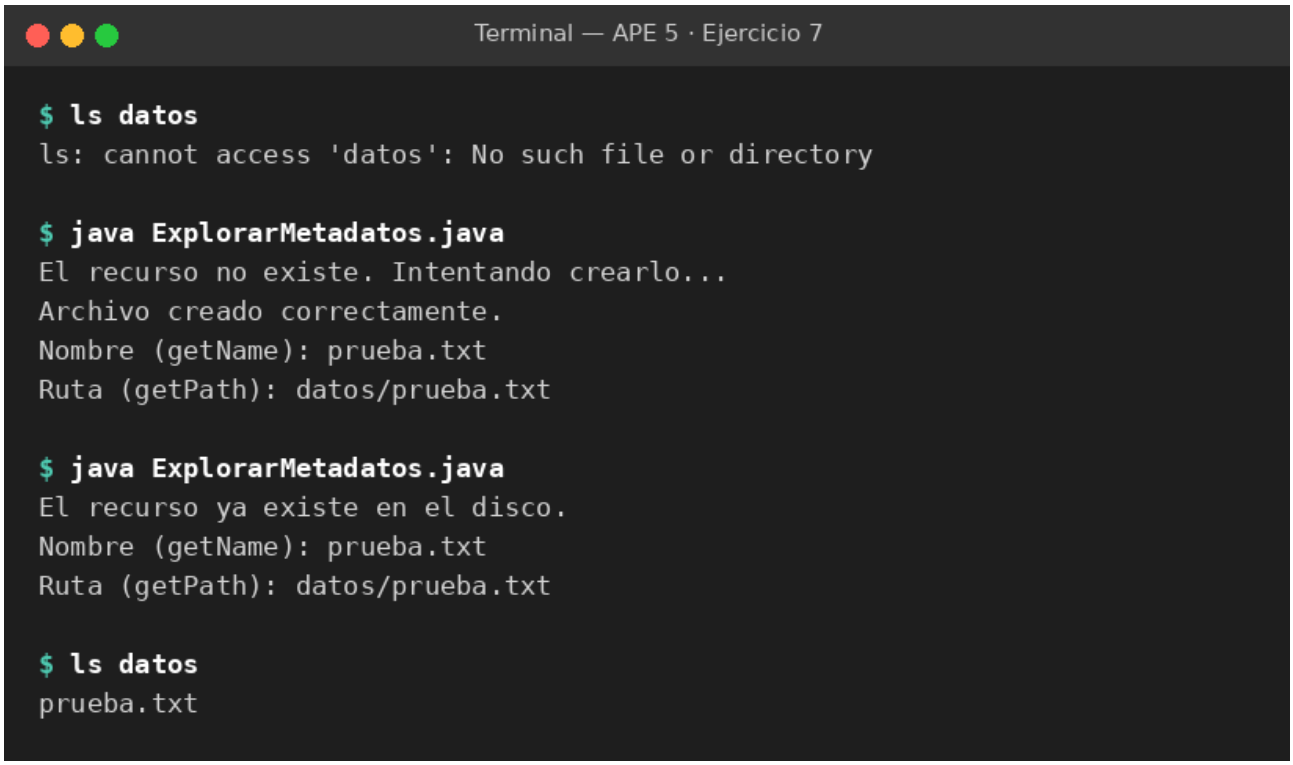
        // 1. Comprobación previa
        if (archivo.exists()) {
            System.out.println("El recurso ya existe en el disco.");
        } else {
            System.out.println("El recurso no existe. Intentando crearlo...");
            // 2. Creación controlada (try-catch obligatorio por IOException)
            try {
                File carpeta = archivo.getParentFile();
                if (carpeta != null && !carpeta.exists()) {
                    carpeta.mkdirs();   // createNewFile() no crea carpetas
                }
                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            } catch (IOException e) {
                System.out.println("Error de E/S al crear el archivo: " + e.getMessage());
            }
        }

        // 3. Extracción de información (siempre se muestra)
        System.out.println("Nombre (getName): " + archivo.getName());
        System.out.println("Ruta (getPath): " + archivo.getPath());
    }
}
```

Prueba de escritorio

Ejecución	exists()	createNewFile()	getName()	getPath()
1ª	false	true (se crea)	prueba.txt	datos/prueba.txt
2ª	true	no se invoca	prueba.txt	datos/prueba.txt

Captura de la ejecución



Ejercicio 8 · Escritura Estructurada por Líneas con BufferedWriter

Parte 3 · Ejercicio 2

Enunciado

Tomando como referencia el flujo de datos desde la memoria RAM hasta el archivo en disco, desarrolla un programa en Java que cree un fichero de texto llamado "materias.txt". El programa debe realizar las siguientes acciones:

1. Configurar un flujo de salida utilizando un objeto BufferedWriter enlazado con un FileWriter.

2. Escribir un listado de cinco asignaturas de tu elección en el archivo.

3. Emplear el método portable newLine() para asegurar que cada asignatura quede guardada estrictamente en una línea independiente.

4. Liberar los recursos de forma correcta invocando el método de cierre (close()) y controlar cualquier error potencial de escritura mediante una excepción de tipo IOException.

5. Verificación: Comprueba físicamente en la carpeta de tu proyecto que el archivo se generó correctamente y que mantiene una única asignatura por línea.

Código (EscribirMaterias.java)

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

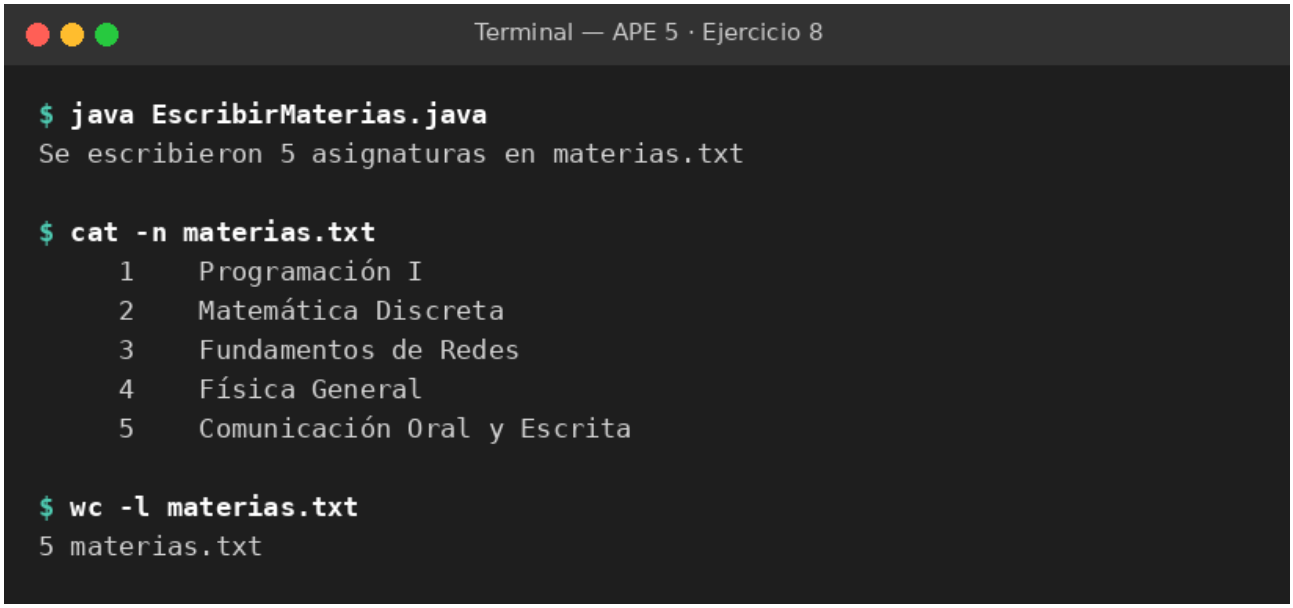
public class EscribirMaterias {
    public static void main(String[] args) {
        String[] materias = {
            "Programación I",
            "Matemática Discreta",
            "Fundamentos de Redes",
            "Física General",
            "Comunicación Oral y Escrita"
        };

        BufferedWriter bw = null;
        try {
            // Flujo: memoria RAM -> BufferedWriter -> FileWriter -> disco
            bw = new BufferedWriter(new FileWriter("materias.txt"));
            for (String materia : materias) {
                bw.write(materia);
                bw.newLine();   // salto de línea portable
            }
            System.out.println("Se escribieron " + materias.length + " asignaturas en materias.txt");
        } catch (IOException e) {
            System.out.println("Error de escritura: " + e.getMessage());
        } finally {
            try {
                if (bw != null) {
                    bw.close();   // libera el recurso y vacía el búfer
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el archivo: " + e.getMessage());
            }
        }
    }
}
```

Prueba de escritorio

Iteración	materia	Acción	Líneas en materias.txt
1	Programación I	write + newLine	1
2	Matemática Discreta	write + newLine	2
3	Fundamentos de Redes	write + newLine	3
4	Física General	write + newLine	4
5	Comunicación Oral y Escrita	write + newLine	5
fin	—	finally → close()	wc -l = 5

Captura de la ejecución



Ejercicio 9 · Control del Modo de Apertura (Append vs Sobrescribir)

Parte 3 · Ejercicio 3

Enunciado

A partir del archivo "materias.txt" generado en el ejercicio anterior, escribe un nuevo programa en Java que demuestre el comportamiento del segundo parámetro en el constructor de FileWriter:

1. Diseña el código de manera que abra el fichero existente configurando el FileWriter en modo de adición (append = true).

2. Añade dos nuevas asignaturas al final del archivo utilizando nuevamente newLine() para no romper el formato de líneas.

3. Asegúrate de cerrar el flujo con .close().

4. Comprobación: Verifica que el contenido previo no se haya sobrescribido y que ahora el archivo conserve las cinco asignaturas originales junto con las dos nuevas incorporadas al final.

Código (AgregarMaterias.java)

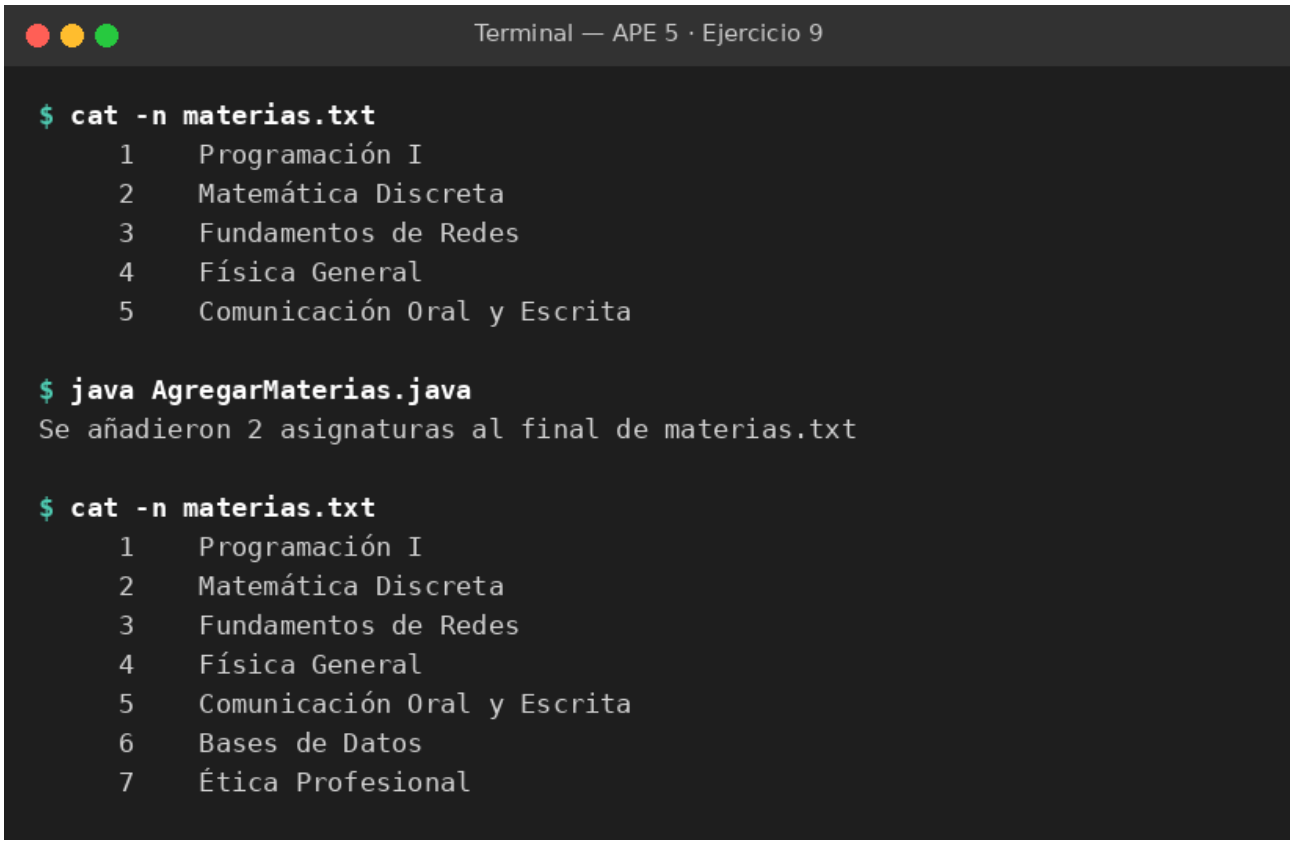
```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class AgregarMaterias {
    public static void main(String[] args) {
        BufferedWriter bw = null;
        try {
            // El segundo parámetro (true) activa el modo APPEND:
            // el contenido anterior se conserva y se escribe al final.
            bw = new BufferedWriter(new FileWriter("materias.txt", true));
            bw.write("Bases de Datos");
            bw.newLine();
            bw.write("Ética Profesional");
            bw.newLine();
            System.out.println("Se añadieron 2 asignaturas al final de materias.txt");
        } catch (IOException e) {
            System.out.println("Error de escritura: " + e.getMessage());
        } finally {
            try {
                if (bw != null) {
                    bw.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el archivo: " + e.getMessage());
            }
        }
    }
}
```

Prueba de escritorio

Momento	Modo	Línea añadida	Total de líneas
Antes	—	—	5 (originales)
write #1	append = true	Bases de Datos	6
write #2	append = true	Ética Profesional	7
Después	close()	—	5 originales intactas + 2 nuevas

Captura de la ejecución



Ejercicio 10 · Ciclo de Vida Completo y Borrado Seguro

Parte 3 · Ejercicio 4

Enunciado

Diseña un script completo en Java que emule el recorrido guiado de gestión de ficheros visto en clase:

1. Construye un objeto File que represente la ruta "datos/temporal.txt".

2. Consulta el estado del archivo mediante exists() y, si no se encuentra disponible, créalo de forma segura controlando la excepción IOException.

3. Implementa una fase de validación en la que declares una variable de control booleana (por ejemplo, boolean confirmarBorrado = true;) para simular la solicitud de confirmación por parte del usuario antes de eliminar datos importantes.

4. Utiliza el método delete() exclusivamente si dicha confirmación es verdadera, imprimiendo por consola un mensaje que refleje el resultado booleano de la operación de eliminación.

Código (CicloVidaFichero.java)

```
import java.io.File;
import java.io.IOException;

public class CicloVidaFichero {
    public static void main(String[] args) {
        // 1. Objeto File con la ruta indicada
        File archivo = new File("datos/temporal.txt");

        // 2. Consulta de estado y creación segura
        if (!archivo.exists()) {
            System.out.println("El archivo no existe. Creándolo...");
            try {
                File carpeta = archivo.getParentFile();
                if (carpeta != null && !carpeta.exists()) {
                    carpeta.mkdirs();
                }
                archivo.createNewFile();
                System.out.println("Archivo creado: " + archivo.getPath());
            } catch (IOException e) {
                System.out.println("Error de E/S: " + e.getMessage());
            }
        } else {
            System.out.println("El archivo ya existe: " + archivo.getPath());
        }

        // 3. Validación: simula la confirmación del usuario
        boolean confirmarBorrado = true;

        // 4. delete() solo si hay confirmación
        if (confirmarBorrado) {
            boolean eliminado = archivo.delete();
            System.out.println("Resultado de delete(): " + eliminado);
        } else {
            System.out.println("Borrado cancelado por el usuario.");
        }
    }
}
```

Prueba de escritorio

Paso	Instrucción	Valor / resultado
1	new File("datos/temporal.txt")	objeto File (el archivo aún no existe)
2	exists()	false → mkdirs() si falta la carpeta + createNewFile() = true
3	boolean confirmarBorrado = true	confirmarBorrado = true
4	if (confirmarBorrado) archivo.delete()	delete() = true → se imprime «Resultado de delete(): true»

Captura de la ejecución

